

CSC1029 OO Programming

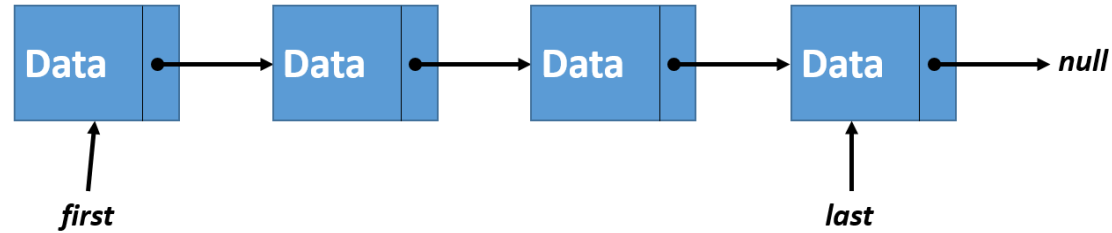
Lecture 26 - Linked Lists & Stacks



What's to come in today's lecture?

- Linked List Behaviour:
 - adding to a specified location
 - removing from a specified location
- Using a Linked List to Implement a Stack

Singly Linked Lists



In the previous lecture we looked at how we can ***add*** to the end and ***remove*** from the end of a singly linked list.

For both of these operations, we need to consider some special cases:

when ***adding***, what happens if the list is ***initially empty***?

when ***removing***, what happens if it's the ***last element***?

In all cases, we need to ensure the ***integrity*** of the structure – i.e. references such as ***first*** and ***last*** are valid etc.

Often it's necessary to reference the ***previous*** item in a list.

Singly Linked Lists

LinkedList

-first:Node

-last:Node

-numElements:int

+LinkedList()

+add(Object)

+add(Object,int):boolean

+remove():Object

+remove(int):Object

+toString():String

+size():int

```
public Object remove(int position) {
    if (numElements == 0 || position < 0 ||
        position >= numElements) {
        return null;
    }
    Object res;
    if (position == 0) {
        // remove from start
    }
    else if (position == numElements - 1) {
        // remove from end
    }
    else {
        // remove from the middle
    }
    this.numElements--;
    return res;
}
```

Singly Linked Lists

LinkedList

-first:Node

-last:Node

-numElements:int

+LinkedList()

+add(Object)

+add(Object,int):boolean

+remove():Object

+remove(int):Object

+toString():String

+size():int

```
Object res;
```

```
if (position == 0) {
```

```
    // case 1: remove from start
```

```
    res = this.first.data;
```

```
    this.first = this.first.next;
```

```
    if (this.first == null)
```

```
        this.last = null;
```

```
}
```

```
else if (position == numElements-1) {
```

```
    // case 2 : remove from end
```

```
    res = remove();
```

```
}
```

```
else {
```

```
    // case 3 : remove from the middle
```

```
    Node previous = select(position - 1);
```

```
    Node target = previous.next;
```

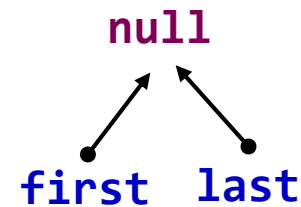
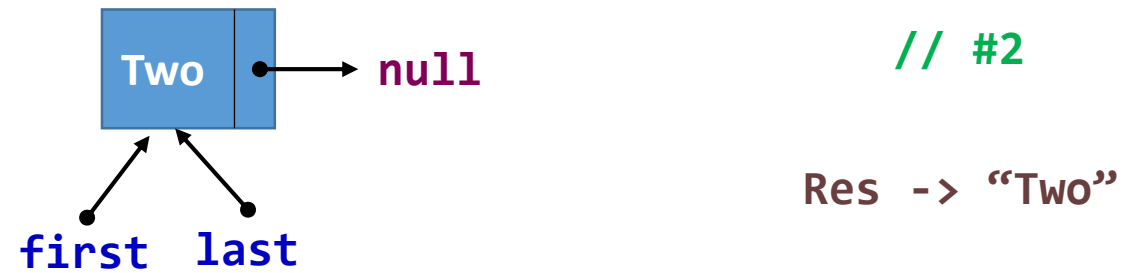
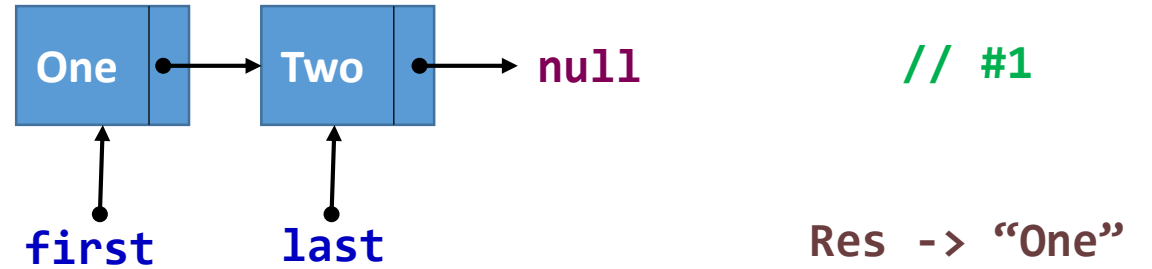
```
    previous.next = target.next;
```

```
    res = target.data;
```

```
}
```

Singly Linked Lists

```
Object res;  
if (position == 0) {  
    // case 1: remove from start  
    res = this.first.data;  
    this.first = this.first.next;  
    if (this.first == null)  
        this.last = null;  
} ...  
  
String str;  
str = (String)myList.remove(0); // #1  
str = (String)myList.remove(0); // #2
```

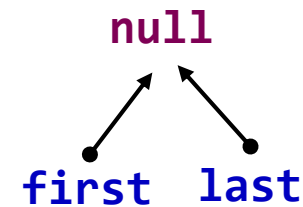
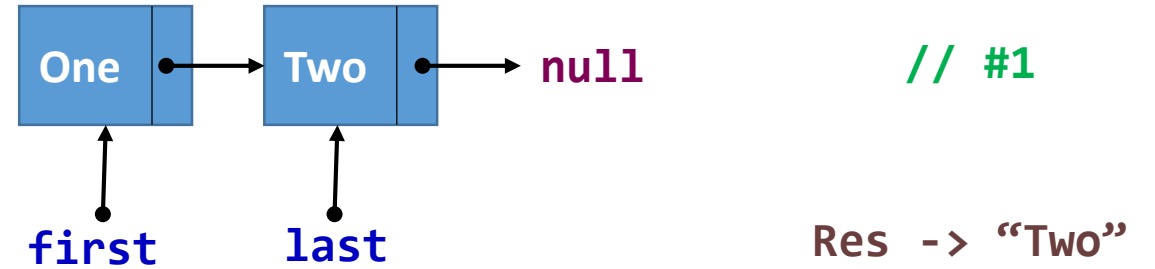


Singly Linked Lists

```
else if (position == numElements-1) {  
    // case 2 : remove from end  
    res = remove();  
}
```

```
String str;  
str = (String)myList.remove(1); // #1  
str = (String)myList.remove(0); // #2
```

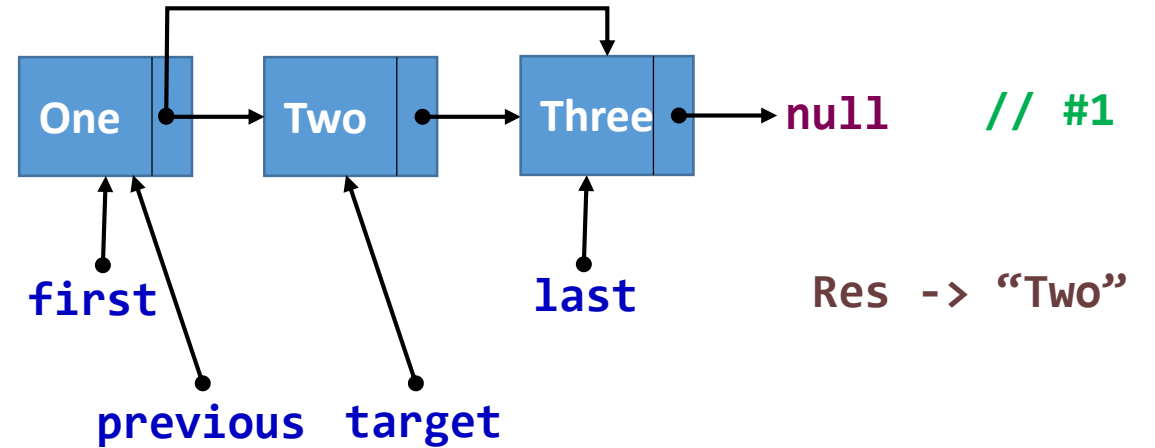
Handled by Case 1



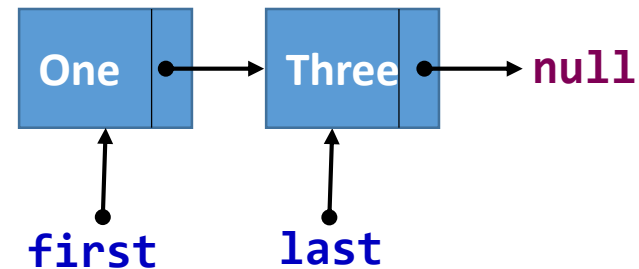
Singly Linked Lists

```
else {  
    // case 3 : remove from the middle  
    Node previous = select(position - 1);  
    Node target = previous.next;  
    previous.next = target.next;  
    res = target.data;  
}
```

```
String str;  
str = (String)myList.remove(1); // #1
```



The `select()` method returns a reference to the list node at the specified position (null if it does not exist) – starts at `first` and skips through the list nodes the required number of times.



Singly Linked Lists

LinkedList

-first:Node

-last:Node

-numElements:int

+LinkedList()

+add(Object)

+add(Object,int):boolean

+remove():Object

+remove(int):Object

+toString():String

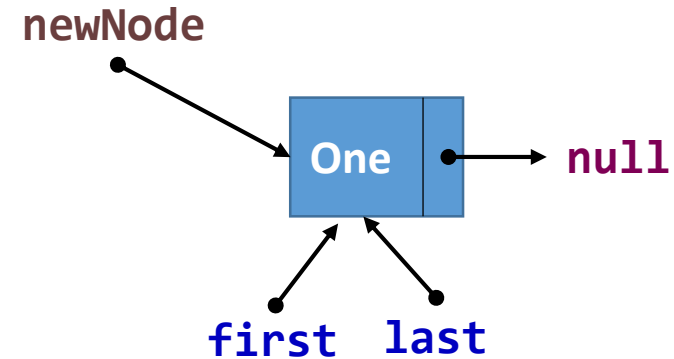
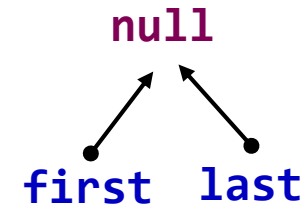
+size():int

```
public boolean add(Object element,
                    int position) {
    if (position >= 0 && position <= numElements &&
        element != null) {
        Node newNode = new Node();
        newNode.data = element;
        if (position == 0) {
            // add at start
        }
        else if (position == numElements) {
            // add at end
        }
        else {
            // add in the middle
        }
        this.numElements++;
        return true;
    }
    return false;
}
```

Singly Linked Lists

```
if (position==0) {  
    // add at start  
    newNode.next = this.first;  
    this.first = newNode;  
    if (this.last == null) {  
        this.last = this.first;  
    }  
}  
else if (position == numElements) {  
    // add at end  
    add(element);  
}  
else {  
    // add in the middle  
    Node temp = select(position-1);  
    newNode.next = temp.next;  
    temp.next = newNode;  
}
```

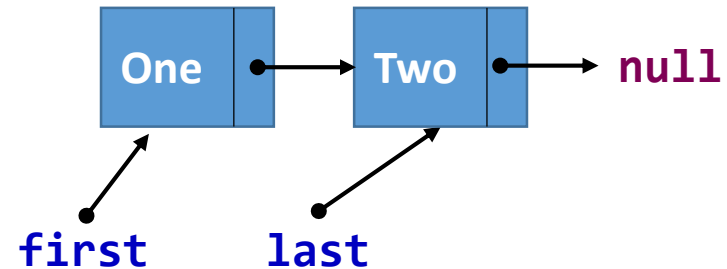
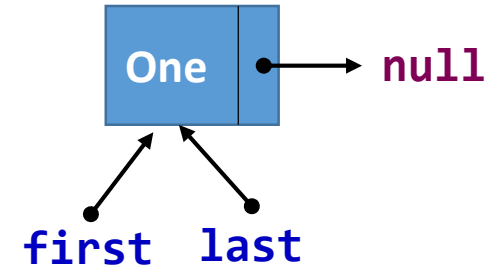
`myList.add("One",0);` // add at start



Singly Linked Lists

```
if (position==0) {  
    // add at start  
    newNode.next = this.first;  
    this.first = newNode;  
    if (this.last == null) {  
        this.last = this.first;  
    }  
}  
else if (position == numElements) {  
    // add at end  
    add(element);  
}  
else {  
    // add in the middle  
    Node temp = select(position-1);  
    newNode.next = temp.next;  
    temp.next = newNode;  
}
```

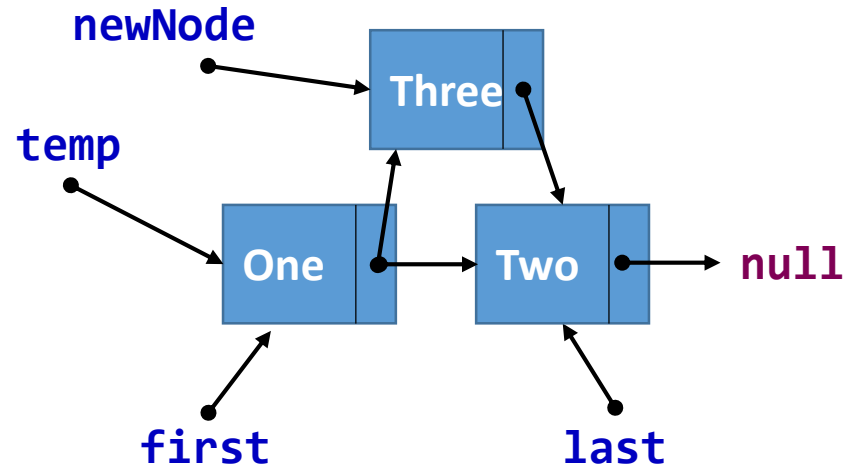
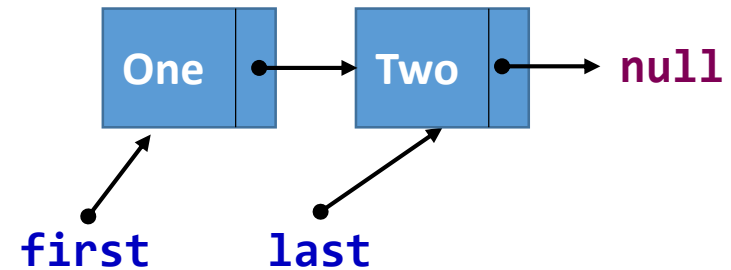
myList.add("Two",1); // add at end



Singly Linked Lists

```
if (position==0) {  
    // add at start  
    newNode.next = this.first;  
    this.first = newNode;  
    if (this.last == null) {  
        this.last = this.first;  
    }  
}  
else if (position == numElements) {  
    // add at end  
    add(element);  
}  
else {  
    // add in the middle  
    Node temp = select(position-1);  
    newNode.next = temp.next;  
    temp.next = newNode;  
}
```

`myList.add("Three",1); // add middle`



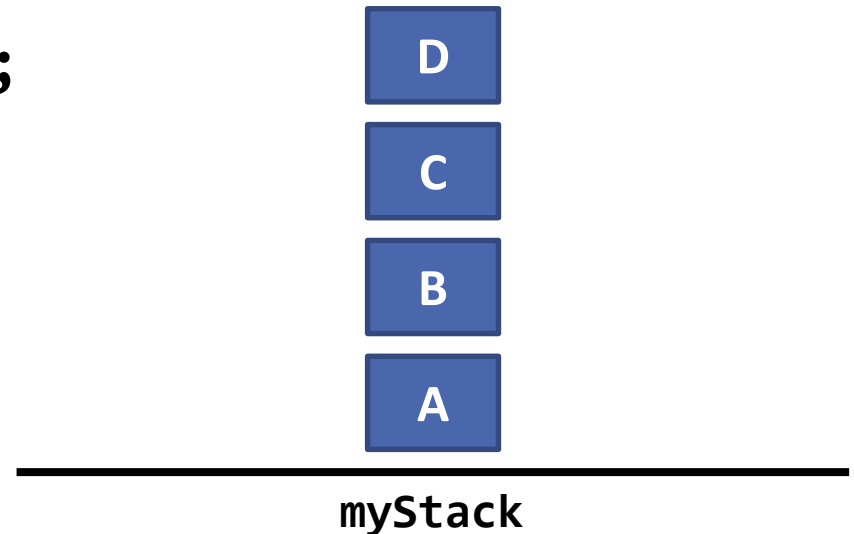
An Introduction to Stacks

A **stack** is a data structure designed to hold data (objects) in a specific order.

In some respects, a stack is similar to a list.

Data is added to (**pushed**) and removed from (**popped**) in a very specific way:

```
// create an empty stack
SimpleStack myStack = new Stack() ;
// add some values
myStack.push("A");
myStack.push("B");
myStack.push("C");
myStack.push("D");
```



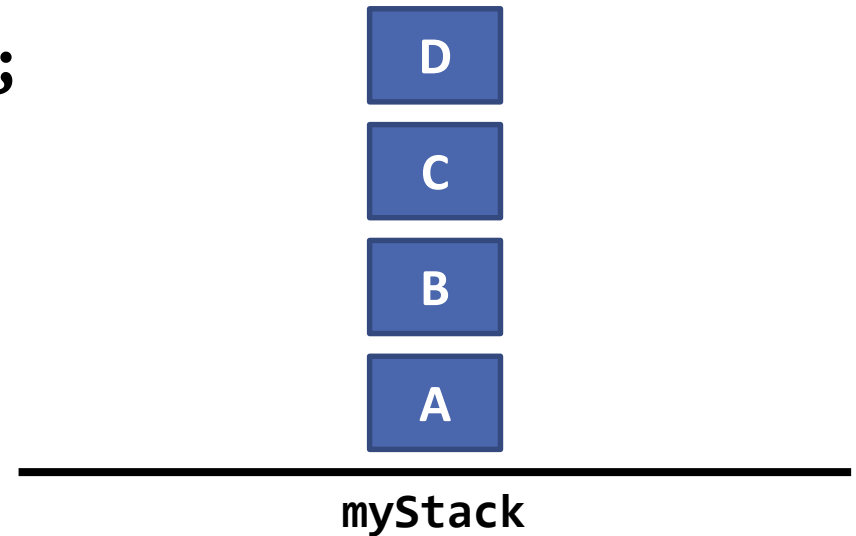
An Introduction to Stacks

A **stack** is a data structure designed to hold data (objects) in a specific order.

In some respects, a stack is similar to a list.

Data is added to (**pushed**) and removed from (**popped**) in a very specific way:

```
// create an empty stack
SimpleStack myStack = new Stack() ;
// remove some values
String str;
str = (String)myStack.pop();
str = (String)myStack.pop();
str = (String)myStack.pop();
str = (String)myStack.pop();
```



A Stack Implementation

SimpleStack
-myList: LinkedList
+SimpleStack() +push(Object):void +pop():Object +isEmpty():boolean

Pop: Five from the stack.
Pop: Four from the stack.
Pop: Three from the stack.
Pop: Two from the stack.
Pop: One from the stack.

```
public class SimpleStack {
    private LinkedList myList;

    public SimpleStack() {
        myList = new LinkedList();
    }

    public boolean isEmpty() {
        return myList.size() == 0;
    }

    public void push(Object data) {
        if (data != null) {
            myList.add(data, 0);
        }
    }

    public Object pop() {
        Object data = myList.remove(0);
        return data;
    }
}
```

```
public static void main(...) {
    SimpleStack myStack =
        new SimpleStack();

    myStack.push("One");
    myStack.push("Two");
    myStack.push("Three");
    myStack.push("Four");
    myStack.push("Five");

    System.out.println();
    String str = (String)myStack.pop();
    while (str != null) {
        System.out.println("Pop: " +
            str + " from the stack.");
        str = (String)myStack.pop();
    }
}
```



What's Next?

More on Stacks
Implementing Queues